

REFLEXIONA-2: Análisis de Dominancia y Contraste Teórico-Experimental

1. ANÁLISIS DE LA OPERACIÓN MÁS COSTOSA

Operación analizada:

agregar_contenido(id, titulo, puntaje)

Suma de complejidades por paso:

Paso 1:

Inserción en Hash Map (Catálogo)

Código: `catalogo[id] = contenido`

Complejidad: $O(1)$

Paso 2:

Recorrido e inserción en Árbol Binario de Búsqueda (BST)

Código: `_bst_insertar(raiz, puntaje, c)`

Complejidad: $O(\log n)$

Paso 3:

Inserción al final del Vector/Pila (Historial)

Código: `historial.push_back(id)`

Complejidad: $O(1)$

Ecuación total:

$T(n) = O(1) + O(\log n) + O(1)$

2. TÉRMINO DOMINANTE

Término dominante:

$O(\log n)$

Justificación:

Aplicando la regla de dominancia para operaciones secuenciales, las inserciones de tiempo constante $O(1)$ (Hash Map y Pila) son absorbidas por el término de mayor crecimiento. El descenso por los niveles del árbol BST es el único paso que escala conforme aumenta "n", por lo que $O(\log n)$ domina el costo total.

3. CONTRASTE: FACTOR TEÓRICO VS MEDIDO

Crecimiento de datos:

De $n = 1,000$ a $n = 10,000$ (Aumento de 10x)

Cálculo del Factor Teórico Esperado (Matemático):

Fórmula: $\log_2(10,000) / \log_2(1,000)$

Operación: $13.28 / 9.96$

Resultado teórico: $\approx 1.33\times$

Cálculo del Factor Medido Real (Benchmark R-6):

Fórmula: $\text{Tiempo}(n=10,000) / \text{Tiempo}(n=1,000)$

Operación: $2.60 \mu\text{s} / 1.80 \mu\text{s}$

Resultado medido: $\approx 1.44\times$

4. CONCLUSIÓN DE CONSISTENCIA

¿Es consistente la teoría con el experimento?

Sí, es altamente consistente.

Explicación:

La matemática Big-O predecía que, a pesar de multiplicar los datos por 10, el tiempo solo crecería un 33% (1.33x). Nuestro experimento mostró un crecimiento del 44% (1.44x). La ligera discrepancia de 0.11x ocurre por factores físicos que Big-O no mide: al tener un árbol de 10,000 nodos, la memoria RAM está más fragmentada. Al saltar entre punteros, el procesador sufre más "Cache Misses" (fallos de caché), añadiendo microsegundos reales al recorrido físico.

IA-REFLEXION-R: Proyección a 1,000,000 de registros

Prompt ingresado a la IA:

"Analiza el siguiente sistema con estas estructuras y complejidades (Hash Map $O(1)$, BST $O(\log n)$, Heap copiado $O(n)$). Si el catálogo creciera a 1,000,000 de registros, ¿qué operación colapsaría primero y cuánto tardaría aproximadamente?"

Predicción de la IA:

"La operación que colapsaría el sistema primero sería la consulta 'Top 5' (procesar_solicitud('top')). Dado que hacen una copia completa de la Cola de Prioridad para evitar vaciarla, la complejidad es $O(n)$. Si $n=10,000$ tomó $\sim 175 \mu\text{s}$ en su benchmark, con 1,000,000 de registros tardaría mínimo $17,500 \mu\text{s}$ (17.5 milisegundos) en pura copia de memoria. Sin embargo, debido al agotamiento de la caché L3 y la recolección de basura / asignación dinámica de memoria, el tiempo real fácilmente superaría los 50-100 ms por consulta. Si muchos usuarios piden el Top 5 simultáneamente, el servidor colapsará por falta de RAM."

Evaluación crítica del equipo:

¿Es realista esta estimación?

Totalmente realista y crítica para el negocio. Asumimos erróneamente que extraer el "Top" de un Heap era siempre eficiente ($O(\log n)$), pero olvidamos que el acto de "clonar" la estructura en C++ o Java demanda recorrer y reasignar memoria para cada uno de los elementos ($O(n)$).

¿Qué aspecto del análisis IA-asistido no consideró el equipo?
No consideramos el costo espacial y de clonación en memoria. Nos enfocamos únicamente en la teoría de "búsqueda" matemática, ignorando que clonar una Cola de Prioridad gigantesca en cada solicitud de un usuario anula por completo la ventaja jerárquica del Heap. La IA nos hizo ver que la solución real para producción sería tener un arreglo separado (caché) pre-calculado $O(1)$ con el Top 5, y actualizarlo solo cuando se inserte una película nueva que supere esos puntajes, en lugar de recalcularlo al momento de la lectura.

// VALIDA V-8: Adaptar lógica de prueba a la API del SistemaMX en C++

```
void ejecutar_casos_valida(SistemaMX& s) {
    cout << "\n=== EJECUTANDO CASOS DE VALIDACIÓN V-8 ===\n";

    // =====
    // CASOS DADOS
    // =====
    // Caso V-1: ID inexistente
    auto r1 = s.procesar_solicitud("lookup", "NO_EXISTE");
    assert(r1.empty() && "V-1: lookup inexistente debe devolver vector vacío");
    cout << "☑ V-1 OK (Lookup inexistente)\n";

    // Caso V-2: Estado inicial y Rango vacío
    auto r2 = s.procesar_solicitud("rango", "", 5.0, 10.0);
    assert(r2.empty() && "V-2: rango en catálogo vacío debe devolver vector vacío");
    cout << "☑ V-2 OK (Rango en sistema vacío no colapsa)\n";

    // =====
    // CASOS PROPIOS (EDGE CASES)
    // =====

    // Inserción de datos de prueba para los Edge Cases
    s.agregar_contenido("MX-10", "Peli A", 8.0);
    s.agregar_contenido("MX-20", "Peli B", 9.0);

    /* CASO PROPIO 1 (V-3): Fronteras matemáticas exactas en BST (Basado en Semana
13)
    Edge Case: Evalúa si el BST incluye películas que caen EXACTAMENTE
    en los límites del rango solicitado. */
    auto r3 = s.procesar_solicitud("rango", "", 8.0, 9.0);
    assert(r3.size() == 2 && "V-3: El rango debe ser inclusivo en las fronteras (<= y >=)");
    cout << "☑ V-3 OK (Fronteras matemáticas exactas)\n";

    /* CASO PROPIO 2 (V-4): Desbordamiento de Top_N (Heap Out of Bounds)
    Edge Case: Solicitar un número de recomendaciones (N=100)
    masivo cuando el catálogo es minúsculo (size=2). */
    auto r4 = s.procesar_solicitud("top", "", 0, 0, 100);
```

```

assert(r4.size() == 2 && "V-4: El heap no debe colapsar al quedarse vacío antes de N");
cout << "☑ V-4 OK (Top_N masivo manejado con seguridad)\n";

/* CASO PROPIO 3 (V-5): Límite Cero (Top_0)
   Edge Case: El usuario (o un bug en la UI) pide el Top 0 recomendaciones. */
auto r5 = s.procesar_solicitud("top", "", 0, 0, 0);
assert(r5.empty() && "V-5: Top 0 debe devolver un arreglo completamente vacío");
cout << "☑ V-5 OK (Solicitud de Top 0 neutra)\n";
}

```

VALIDA: Reporte de Casos de Prueba de Flujos Completos

=====

CASO DADO 1: Búsqueda Lookup de ID Inexistente

=====

Descripción: Se solicita al enrutador (Hash Map) un ID ("NO_EXISTE").
 Resultado esperado: Retornar vector vacío, sin excepciones de puntero.
 Resultado real: Vector vacío.
 Estado: ☑ PASA

=====

CASO DADO 2: Consulta de Rango en Estado Vacío (Zero-State)

=====

Descripción: Se solicita un rango válido antes de que haya datos en el sistema.
 Resultado esperado: El BST detecta `nodo == nullptr` y retorna vector vacío.
 Resultado real: Vector vacío sin 'Segmentation Fault'.
 Estado: ☑ PASA

=====

CASO PROPIO 1 (Edge Case): Fronteras matemáticas exactas en BST

=====

(a) Descripción:
 Basado en el error lógico descubierto en la Semana 13 (IA-REFLEXION-V). Se busca el rango exacto entre 8.0 y 9.0 existiendo películas con puntuación de 8.0 y 9.0 cerrados.
 Relevancia: Prueba que la implementación use comparadores inclusivos ($\geq$ y $\leq$). Si usa ($<$ y $>$), los datos en el borde son descartados erróneamente.
 (b) Resultado esperado: Devuelve ambos elementos (tamaño del vector = 2).
 (c) Resultado real: Vector contiene los 2 elementos exactos.
 (d) Estado: ☑ PASA (Se corrigió la lógica a $\leq$ y $\geq$ durante S13).

=====

CASO PROPIO 2 (Edge Case): Desbordamiento de Top_N (Heap Emptying)

=====

(a) Descripción:
 Se piden las "Top 100 recomendaciones" pero el sistema solo tiene 2 películas registradas.
 Relevancia: Un edge case crítico para las colas de prioridad. Si el bucle while del Enrutador solo evalúa `contador < 100` e ignora `!cola.empty()`, el sistema intentará hacer `pop()` a un Heap vacío, colapsando el programa completo.
 (b) Resultado esperado: Devuelve las 2 películas existentes y termina pacíficamente.
 (c) Resultado real: Devuelve vector de tamaño 2.

(d) Estado: ☒ PASA.

=====

CASO PROPIO 3 (Edge Case): Consulta Neutral de N=0 (Top 0)

=====

(a) Descripción:

El usuario solicita el "Top 0".

Relevancia: Falla de UI o API común. El sistema no debe iterar y no debe arrojar valores negativos. Valida la condición estricta de arranque del iterador.

(b) Resultado esperado: Vector vacío devuelto instantáneamente, O(1) tiempo real.

(c) Resultado real: Vector vacío, no entró al ciclo del Heap.

(d) Estado: ☒ PASA.

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <random>

// Funciones auxiliares para verificar invariantes
int contar_nodos_bst(NodoBST* nodo) {
    if (nodo == nullptr) return 0;
    return 1 + contar_nodos_bst(nodo->izq) + contar_nodos_bst(nodo->der);
}

void inorder_recorrido(NodoBST* nodo, vector<double>& resultado) {
    if (nodo != nullptr) {
        inorder_recorrido(nodo->izq, resultado);
        resultado.push_back(nodo->clave);
        inorder_recorrido(nodo->der, resultado);
    }
}

void verificar_invariante_cruzada(SistemaMX& sistema) {
    cout << "\n=== VERIFICANDO INVARIANTE CRUZADA ===" << endl;
    vector<string> errores;

    int n_dict = sistema.catalogo.size();
    int n_bst = contar_nodos_bst(sistema.bst_raiz);
    int n_hist = sistema.historial.size();

    // Inv-1: dict vs BST
    if (n_dict != n_bst) {
        errores.push_back("Inv-1 ✖: dict tiene " + to_string(n_dict) +
            " pero BST tiene " + to_string(n_bst) + " nodos");
    }

    // Inv-2: historial vs dict
    if (n_hist != n_dict) {
        errores.push_back("Inv-2 ✖: historial tiene " + to_string(n_hist) +
```

```

        " pero dict tiene " + to_string(n_dict));
    }

    // Inv-3: Orden del BST
    vector<double> inorder_result;
    inorder_recorrido(sistema.bst_raiz, inorder_result);

    // Validar si está ordenado de forma ascendente
    if (!is_sorted(inorder_result.begin(), inorder_result.end())) {
        errores.push_back("Inv-3 ✗: BST no está en orden ascendente");
    }

    if (errores.empty()) {
        cout << "☑ Invariante cruzada OK: " << n_dict << " elementos en todos los índices"
        << endl;
    } else {
        for (const auto& e : errores) cout << e << endl;
    }
}

// Ejecución de 50 operaciones
void ejecutar_prueba_50() {
    SistemaMX s;
    random_device rd;
    mt19937 gen(rd());
    uniform_real_distribution<> dis(1.0, 10.0);

    for (int i = 0; i < 50; i++) {
        string id = "ID" + to_string(i);
        string titulo = "Titulo_" + to_string(i);
        double puntaje = dis(gen);
        s.agregar_contenido(id, titulo, puntaje);
    }

    verificar_invariante_cruzada(s);
}

```

VALIDA: Reporte de verificar_invariante_cruzada (V-9)

```

=====
REPORTE DE INVARIANTES TRAS 50 OPERACIONES ALEATORIAS
=====

```

Invariante 1 (dict == BST): ☒ PASO

Explicación: El catálogo Hash tiene exactamente 50 elementos y la función `contar\_nodos\_bst` retornó 50 nodos. Esto demuestra que no hay pérdida de datos entre estructuras al realizar inserciones atómicas mediante `agregar\_contenido`.

Invariante 2 (hist == dict): ☒ PASO

Explicación: El historial (Pila/Vector) tiene un tamaño de 50, coincidiendo perfectamente con el número de elementos únicos en el Hash Map. Cada registro tiene trazabilidad.

Invariante 3 (BST en orden): ☒ PASO

Explicación: El recorrido `in-order` devolvió una lista con los 50 puntajes flotantes en orden estrictamente ascendente, verificando que la propiedad estructural (Invariante BST de la Semana 13) se mantuvo estable para cada nodo (menores a la izquierda, mayores a la derecha).

ANÁLISIS DE ESTABILIDAD Y POSIBLES FALLOS

Aunque todas nuestras invariantes pasaron (☒) durante el desarrollo temprano identificamos una situación donde Inv-1 (dict == BST) podría fallar (☒)

Si el generador aleatorio produjera dos películas con el MISMO PUNTAJE exacto (ej. 8.5 y 8.5) y nuestra inserción de BST usara condiciones estrictas (`<` y `>`) omitiendo el `=`, el BST descartaría la segunda película. Esto provocaría que el dict tuviera 50 elementos, pero el BST tuviera 49, rompiendo el sistema.

Nuestra corrección preventiva (implementada en el reto APLICA-4 basado en la Semana 13) fue usar `>=` y `<=` en el algoritmo del árbol o delegar el empate al hijo derecho, garantizando que el BST siempre guarde la misma cantidad de datos que el diccionario, incluso con puntajes duplicados.

IA-REFLEXION-V:

Respuesta de la IA sobre los falsos positivos: Sí, existen escenarios de uso real donde esta verificación daría falsos positivos. El punto ciego de la función verificar_invariante_cruzada es que confía exclusivamente en la "coincidencia de conteos" (longitud) y en el ordenamiento interno del árbol, pero nunca verifica la "identidad del contenido". Un escenario real de falso positivo ocurre si el sistema sufre una desincronización donde se inserta la película "ID001" en el diccionario, pero por un error de paso de parámetros se inserta "ID999" en el BST. El diccionario crece en 1, el árbol crece en 1 y el historial crece en 1. Las invariantes 1 y 2 pasan. El BST ordena su contenido correctamente y la invariante 3 pasa. El sistema reporta ☒ OK, pero las estructuras de datos tienen elementos completamente distintos.

Evaluación del equipo: ¿Es un escenario realista para el proyecto StreamMX? Es un escenario altamente realista. Basado en los retos que enfrentamos en las semanas previas (como los errores lógicos con los rangos de puntajes, fronteras de datos y el manejo de

tipos de datos en la Semana 12 y 13), es muy fácil que al actualizar una película existente, el diccionario sobrescriba la clave (manteniendo el mismo tamaño), mientras que el BST, si no está bien balanceado o programado para actualizar, inserte un nodo duplicado. Si por casualidad otra operación falla y se omite una inserción en el diccionario, los tamaños absolutos (n_{dict} y n_{bst}) podrían coincidir matemáticamente por pura casualidad, ocultando la corrupción de los datos para la demo del Boss.

Fin Parte Mane